

LLM Reasoning over Evolving Knowledge Graphs

Anonymous Author(s)*

ABSTRACT

Large language models (LLMs) increasingly rely on external knowledge graphs (KGs) for factual reasoning, yet existing approaches typically assume that KGs are static. In practice, knowledge is incomplete, continuously evolves, and accumulates conflicting updates, requiring reasoning and knowledge maintenance to be addressed jointly rather than independently. We study *reasoning over evolving knowledge*, where an LLM must plan under graph incompleteness, ground evidence under temporal ambiguity, and update the KG as new information arrives.

We propose a unified, training-free framework that connects evolving external knowledge with structured LLM reasoning. The framework combines **EvoReasoner**, which performs bounded multi-route reasoning with context- and temporally informed exploration to recover from missing or ambiguous graph paths, and **EvoKG**, which aligns knowledge extracted from incoming documents while preserving temporal context and resolving conflicting facts. The framework improves knowledge utilization without retraining the underlying LLM. We evaluate our approach on temporal question answering benchmarks and an end-to-end setting where raw documents provide missing or updated knowledge. Across model families and scales, it consistently outperforms prompting-only, retrieval-based, and graph-reasoning baselines. For example, an 8B LLM improves from 18.6% without graph support to 37.0% after reasoning over the evolved KG, showing that jointly evolving external knowledge and reasoning over it can substantially improve factual question answering. We release our code and data at anonymous.4open.science/r/TREK-434C.

ACM Reference Format:

Anonymous Author(s). 2018. LLM Reasoning over Evolving Knowledge Graphs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Large language models (LLMs) [1] increasingly support knowledge-intensive tasks in science, medicine, finance, and other domains where users expect factual answers supported by multi-step reasoning [6, 17, 28, 29]. However, the world changes continuously, while factual knowledge stored in LLM parameters remains largely fixed after pretraining and is costly to refresh [8, 30]. Retrieval-augmented generation addresses this limitation by connecting LLMs to external knowledge sources such as knowledge graphs

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

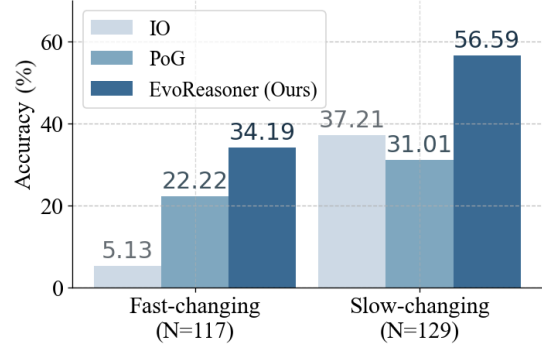


Figure 1: QA accuracy on CRAG questions grouped by the required knowledge type. Fast-changing questions expose outdated parametric knowledge, while slow-changing questions expose noise and temporal ambiguity in graph retrieval.

(KGs), which provide structured entities, relations, and multi-hop reasoning paths [2, 14, 15, 19, 23, 24, 27]. Yet external knowledge is itself not static. KGs must incorporate new information while preserving historical facts and resolving conflicting updates, making knowledge maintenance as important as knowledge retrieval [8, 30]. To sustain the effectiveness of knowledge-augmented LLMs, the graph itself must evolve as new information arrives. Although KGs can be maintained through human curation or conventional pipelines, LLMs offer a flexible mechanism for extracting and aligning new information at scale [5, 18, 35]. We call this setting *reasoning over evolving knowledge*: given a factual question and an external KG whose evidence may be incomplete, stale, or conflicting, a system must ground and reason over the current graph while incorporating incoming evidence to maintain it.

Figure 1 illustrates two complementary failure modes on CRAG, a factual QA benchmark that pairs questions with mock domain-specific KGs and distinguishes questions by the dynamism of their required knowledge [34]. For fast-changing questions, required facts can postdate model pretraining: direct prompting reaches 5.13%, while the graph-based knowledge reasoning method Plan-on-Graph (PoG) [2] reaches 22.22% by exposing external evidence but remains vulnerable to missing or temporally ambiguous paths. For slow-changing questions, direct prompting is already competitive (37.21%), yet indiscriminate graph retrieval introduces irrelevant or contradictory evidence and reduces PoG to 31.01%. Our method reaches 34.19% and 56.59% in the two regimes, respectively. Thus, effective reasoning over evolving knowledge requires both a well-maintained graph and a reasoning process that adapts to its incompleteness and temporal ambiguity.

We argue that reasoning over evolving knowledge requires addressing three coupled challenges. First, **planning under graph incompleteness**: existing graph-reasoning and temporal retrieval systems often rely on an LLM-generated retrieval plan or path over the KG [2, 19, 24, 27]. Such plans can be effective on clean graphs, but in an incomplete KG the intended path may simply not exist.

A reasoner that commits to one route can fail even when an alternative route through related entities would recover the answer. Second, **grounding under temporal ambiguity**: the same surface mention may refer to multiple entities or facts indexed by different times. A query about an award, team, role, or affiliation often requires selecting evidence compatible with the query’s temporal context rather than the most semantically similar neighbor. Third, **evolution under noisy updates**: KG construction and editing pipelines can add newly extracted facts, but directly replacing old tail entities with new ones risks corrupting the graph when the extraction is noisy or when multiple values are valid under different temporal or regional contexts [5, 18, 25, 35].

We introduce a unified, training-free framework for LLM reasoning over evolving KGs (Figure 2). The framework jointly addresses the three challenges above. **EvoREASONER** improves reasoning under graph incompleteness through bounded multi-route planning, context-informed grounding, and temporally informed local exploration. Rather than committing to a single reasoning path, it adaptively explores multiple plausible routes and progressively refines the search frontier. Complementarily, **EvoKG** evolves the KG by extracting and aligning knowledge from incoming documents, distinguishing node-level properties from time-stamped relations, preserving historical evidence, and maintaining confidence-aware competing property values. Together, the two components connect an evolving external memory to a structured reasoning workflow.

The framework specifies the control flow and interfaces between planning, grounding, exploration, aggregation, and graph update operators. These operators define the reasoning workflow independently of any particular implementation. In our implementation, semantic operators are instantiated with LLM prompts and candidate retrieval uses off-the-shelf semantic matching. This design keeps the system training-free: it can use newly available documents and updated KGs without retraining the underlying LLM or optimizing task-specific policies. It targets practical LLM reasoning over evolving knowledge by explicitly structuring planning, grounding, exploration, and knowledge evolution.

We evaluate the approach on two temporal KGQA benchmarks and on an end-to-end setting in which raw documents provide missing or updated facts. **EvoREASONER** improves accuracy by up to 14.0% over prior KG-based reasoning methods. In the single-stage CRAG enrichment setting, moving **EvoREASONER** from the original to the evolved KG improves accuracy by 1.36–8.69% across the reported domain–model pairs. The gains hold across model families and scales. For example, LLaMA 3.1–8B improves from 18.6% without graph support to 37.0% with the evolved KG, showing that improving the quality and utilization of evolving external knowledge can strengthen downstream reasoning without modifying the underlying LLM parameters.

Our contributions are:

- **A problem formulation and workflow for reasoning over evolving knowledge.** We identify planning under graph incompleteness, grounding under temporal ambiguity, and knowledge evolution under conflicting updates as three coupled requirements, and instantiate them in **EvoREASONER** and **EvoKG**.
- **Structured reasoning under incomplete and evolving knowledge.** **EvoREASONER** combines bounded multi-route planning,

context-informed grounding, and temporally informed local exploration to recover from missing reasoning paths while suppressing temporally irrelevant search branches.

- **Incremental KG evolution with temporal context.** **EvoKG** aligns facts extracted from incoming documents, distinguishes node-level properties from time-stamped relations, retains historical context, and combines frequency with temporal decay when scoring competing property values.
- **Broad empirical evaluation.** Experiments across four benchmarks, five LLMs, intrinsic KG construction, ablations, and efficiency analyses show consistent gains over LLM-only, retrieval, and graph-reasoning baselines.

2 PRELIMINARIES

Definition 2.1 (Attributed Knowledge Graph). An attributed knowledge graph is a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R}, \mathcal{P})$, where each node $v \in \mathcal{V}$ and each edge $e \in \mathcal{E}$ has a type given by $\tau(v) : \mathcal{V} \rightarrow \mathcal{A}$ and $\phi(e) : \mathcal{E} \rightarrow \mathcal{R}$, respectively. In addition, each node and edge is associated with a property map $\pi : \mathcal{V} \cup \mathcal{E} \rightarrow \mathcal{P}$, where $\mathcal{P}$ denotes the space of key-value pairs. They allow nodes and edges to store arbitrary attributes in the form of a hashmap.

Problem Formulation. Let $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \pi_t)$ denote the KG at time t , where π_t stores attribute metadata for nodes and edges, including temporal information. Each edge $e \in \mathcal{E}_t$ may be associated with a temporal validity interval $[\tau_{\text{start}}(e), \tau_{\text{end}}(e)] \subseteq \mathbb{R} \cup \{\perp\}$, where $\perp$ denotes an unknown or open-ended temporal boundary, indicating when the associated fact holds true. These start and end times are stored in the property map: $\tau_{\text{start}}(e) = \pi_t(e)[\text{valid from}]$, $\tau_{\text{end}}(e) = \pi_t(e)[\text{valid until}]$.

Given a natural language question x , potentially with an explicit or implicit temporal scope (e.g., an explicit temporal question could be “Who was the president of the U.S. in 2008?”, while an implicit one could be “Who was the president when the financial crisis happened?”), the goal is to identify a set of entities $\mathcal{A} \subseteq \mathcal{V}_t$ by reasoning over the temporally relevant subgraph of $\mathcal{G}_t$. These entities are then used to formulate the final response.

3 STRUCTURED REASONING OVER EVOLVING KGS

EvoREASONER is a structured inference workflow that operationalizes two principles from Section 1: planning under graph incompleteness and grounding under temporal ambiguity. As illustrated in Figure 2, it comprises (1) **bounded multi-route decomposition**, (2) **context-informed global initialization**, and (3) **temporally informed local exploration**. In our implementation, planning and relevance-scoring operators are instantiated through the LLM prompts in Appendix D, while the workflow keeps the interfaces among planning, grounding, exploration, and answer aggregation explicit.

3.1 Multi-Route Decomposition

We first ask the planner to decompose query x into several semantic routes (e.g., Routes A and B in Figure 2). The number of routes is bounded by a fixed generation budget rather than chosen without constraint by the LLM; invalid and semantically redundant routes are removed before traversal. Alternative routes are useful when

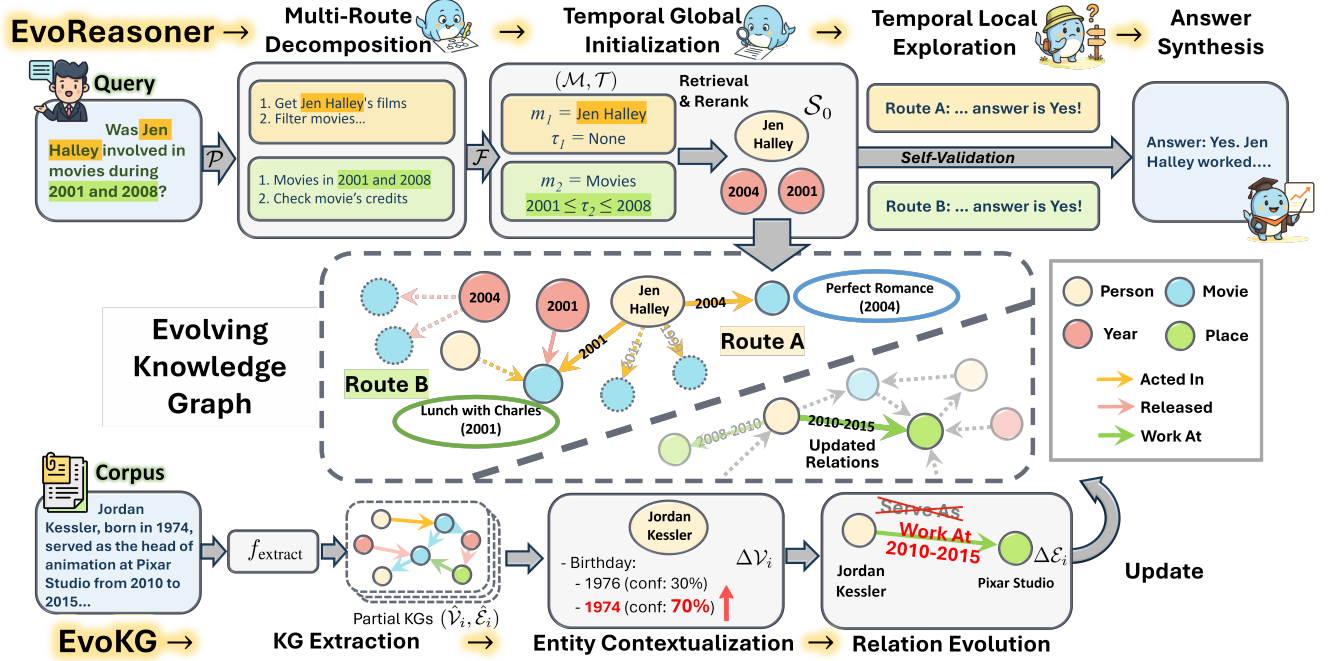


Figure 2: The overall framework, consisting of two components: EvoReasoner for multi-hop temporal reasoning and EvoKG for document-driven KG evolution. The top pipeline shows how a query is decomposed, grounded to temporal entities, explored locally, and synthesized into an answer. The bottom pipeline illustrates how external documents update the KG via entity and relation contextualization with confidence-aware resolution and temporal evolution.

an entity is missing or incompletely linked. For example, the Jen Halley question in Figure 2 can be answered either by starting from the actor and filtering movies by year, or by retrieving movies in the requested years and then checking the cast. If the actor node cannot be grounded, the second route provides a fallback. We prioritize concise routes and expand alternatives only when necessary.

Let $\mathcal{R} = \{r_1, \dots, r_J\}$ denote the set of candidate decomposition routes, where each route r_j consists of a sequence of subgoals $r_j = \{x_1^{(j)}, x_2^{(j)}, \dots, x_{T_j}^{(j)}\}$, where T_j is the length of route r_j . The framework denotes the route-planning operator by $\mathcal{P}$; in our implementation it is an LLM prompted with the template in Appendix D. It maps the query to candidate plans $x \xrightarrow{\mathcal{P}} \mathcal{R}$. To prioritize routes, we use the following scalar heuristic:

$$\text{Cost}(r_j) = \sum_{t=1}^{T_j} \psi(x_t^{(j)}), \text{ where } \psi(x_t^{(j)}) = (b_t \cdot n_t)^{h_t}.$$

$\psi(x_t^{(j)})$ is a lightweight planning heuristic rather than an exact graph-traversal cost. The planner estimates b_t (expected branching), n_t (approximate reachable search space), and h_t (expected hops) from the subgoal structure and, when available, coarse KG statistics. It does not require enumerating the current entity or relation sets, nor does it require numerically precise estimates to be useful: it only needs to favor clearly narrower routes. For example, starting from Jen Halley and exploring her film edges is evidently narrower than enumerating all movies in the KG and checking their casts. If the relevant Jen-Halley-to-film links are missing, the broader route remains a retained fallback. We retain the top- N lowest-cost and

semantically distinct plans:

$$\mathcal{R}^* = \{r_j \mid (r_j, -\text{Cost}(r_j)) \in \text{TopK}_N(\{(r_\ell, -\text{Cost}(r_\ell)) \mid r_\ell \in \mathcal{R}\})\}.$$

These selected routes are used to guide the traversal process, offering complementary reasoning strategies that can be aggregated for answer synthesis and justification.

3.2 Context-Informed Global Initialization

Given the decomposed routes, our method begins by identifying topic entities in $\mathcal{G}_t$ that are semantically and temporally aligned with the query x and its subgoals $\{x_t^{(j)}\}$. Prior works (e.g., Sun et al. [27], Yang et al. [34]) typically extract topic entities directly using simple string matching, without verifying their presence in the KG or accounting for the temporal context. While this may suffice in static and clean KGs, it fails in evolving and noisy KGs. For example, a query mentioning “Pixar Company” cannot be grounded if the KG only contains the entity “Pixar Animation Studio.” Similarly, “Oscar Awards” lacks temporal specificity unless linked to one of its 100 historical variants.

To address these challenges, we propose a *context-aware global initialization* strategy that leverages both semantic similarity and temporal signal. This ensures robust grounding to valid entities, even in cases of naming ambiguity or temporally-indexed duplicates, and avoids the common failure case where reasoning begins from non-existent or irrelevant nodes. Let $\mathcal{F}$ represent a query analysis function that produces two aligned outputs, where $\mathcal{M}$ is

a disjoint set of entity mentions in the query and $\mathcal{T}$ is the corresponding set of temporal contexts (e.g., “between 2001 and 2008” in Figure 2):

$$\mathcal{F}(x) \rightarrow (\mathcal{M}, \mathcal{T}), \mathcal{M} = \langle m_1, \dots, m_n \rangle, \mathcal{T} = \langle \tau_1, \dots, \tau_n \rangle.$$

Each pair (m_i, τ_i) is normalized into a structured textual form and embedded as $\mathbf{h}_{m_i}^{\tau_i} = \text{Enc}(m_i, \tau_i)$. In our implementation, $\text{Enc}(\cdot)$ is instantiated with off-the-shelf semantic matching over timestamp-augmented text, so that entity candidates can be compared under the query’s temporal context. This embedding is then matched against all KG entity embeddings $\{\mathbf{h}^{(v)} \mid v \in \mathcal{V}_t\}$ via an efficient vector search. To retrieve grounding candidates, we use cosine similarity to select the top- k most similar nodes:

$$C_i = \text{TopK}_k(\{v \in \mathcal{V}_t : \text{sim}(\mathbf{h}_{m_i}^{\tau_i}, \mathbf{h}_v)\}),$$

where $\text{TopK}_k(\cdot)$ returns the k highest-scoring entities under cosine similarity $\text{sim}(\cdot, \cdot)$, and k is a predefined hyperparameter (e.g., $k = 5$).

Each retrieved candidate $v \in C_i$ is further verified using a relevance scoring function (see Appendix D for the prompt template), $s_{\text{init}}(v \mid x, \{x_\ell^{(j)}\}_{\ell=1}^{T_j}, \tau_i) \in [0, 1]$, which scores whether a KG entity v is an appropriate grounding for m_i , conditioned on the full question x and its associated temporal context τ_i . This allows for disambiguation among semantically similar or temporally conflicting entities. Let w , the reasoning width, be the number of final candidates retained per entity mention in the query. The final set of anchor entities (frontier nodes at step 0) is selected as

$$\mathcal{S}_0 = \bigcup_i \text{TopK}_w(\{v \in C_i : s_{\text{init}}(v \mid x, \{x_\ell^{(j)}\}_{\ell=1}^{T_j}, \tau_i)\}).$$

3.3 Temporally Informed Local Exploration

After global initialization, our approach performs beam search-based local exploration to traverse the KG. At each step, neighboring relations and entities are scored for relevance to the query. Temporal context is particularly helpful in domains like sports or politics, where entities are linked to numerous time-stamped events; without it, exploration can follow semantically similar but temporally mismatched evidence.

Local exploration scores neighboring facts using semantic content together with textualized temporal validity. This temporal refinement suppresses semantically similar but temporally irrelevant branches and concentrates exploration on the portion of the graph most compatible with the current subgoal. Let $\mathcal{X}^{(j)} = \{x_\ell^{(j)}\}_{\ell=1}^{T_j}$ denote the subgoals of route r_j . At step i , given current entity $v \in \mathcal{S}_i$, we perform three actions:

1. Relation Selection: We retrieve all outgoing relations $r \in \mathcal{R}_i(v)$ and the relevance scoring function $s_{\text{rel}}(r \mid x, \mathcal{X}^{(j)})$ scores them for relevance to the current query and subgoals. The top-ranked relations are selected as follows:

$$\mathcal{R}_i^{\text{sel}} = \text{TopK}_w(\{r \in \mathcal{R}_i(v) : s_{\text{rel}}(r \mid x, \mathcal{X}^{(j)})\}),$$

where w is the reasoning width.

2. Entity Expansion: For each relation $r \in \mathcal{R}_i^{\text{sel}}$, we retrieve connected triplets of the form $(v, r, v') \in \mathcal{E}_t$, forming a set of candidate triplets: $\mathcal{T}_i = \{(v, r, v') \in \mathcal{E}_t : r \in \mathcal{R}_i^{\text{sel}}\}$.

3. Temporal Evidence Selection: Each triplet $(v, r, v') \in \mathcal{T}_i$ is verbalized (e.g., “Team A played Team B in 2020”) together with its validity interval $[\tau_{\text{start}}, \tau_{\text{end}}] \subseteq \mathbb{R} \cup \{\perp\}$, where $\perp$ denotes unknown time. We compute $s_{\text{triplet}}(x, (v, r, v')) = \text{sim}(\text{Enc}(x), \text{Enc}(v, r, v', \tau))$ and retain the highest-scoring triplets. For a question about the 2012 Oscar Awards, this favors 2012 award entities over otherwise similar events from other years; for questions with weak temporal constraints, its contribution is expected to be smaller.

$$\mathcal{T}_i^{\text{sel}} = \text{TopK}_w(\{t \in \mathcal{T}_i : s_{\text{triplet}}(x, (v, r, v'))\}),$$

$$\mathcal{S}_{i+1} = \{v' \mid (v, r, v') \in \mathcal{T}_i^{\text{sel}}\}.$$

The destination entities in $\mathcal{S}_{i+1}$ from the selected triplets define the next search frontier. The reasoning proceeds until either (1) a stopping criterion is met (i.e., the answer is found or the maximum depth is reached), or (2) no high-confidence edges are available for expansion.

3.4 Answer Synthesis and Justification

We synthesize a final answer by aggregating high-scoring paths. Each path $\gamma = \{(r_1, v_1), \dots, (r_L, v_L)\}$, starting from anchor $v \in \mathcal{S}_0$, is assigned an aggregation weight:

$$W(\gamma) = s_{\text{init}}(v) \prod_{i=1}^L [s_{\text{rel}}(r_i \mid x, \mathcal{X}^{(j)}) \cdot s_{\text{triplet}}(x, (v_{i-1}, r_i, v_i))].$$

where $s_{\text{init}}(\cdot)$ is the global initialization score from Section 3.2, $s_{\text{rel}}(\cdot)$ scores relation relevance at step i , and $s_{\text{triplet}}(\cdot)$ captures the temporal and semantic alignment of the traversed edge. The product serves as a relative aggregation weight for combining paths.

Majority Voting Across Reasoning Routes. Each route in the top-ranked reasoning routes $\mathcal{R}^*$ yields a predicted answer $a_i \in \mathcal{V}_t$ with aggregation weight $W(\gamma_i)$. The final answer is selected via weighted voting: $a^* = \arg \max_{a \in \mathcal{A}} \sum_{i: a_i=a} W(\gamma_i)$. Weighted voting consolidates predictions from multiple plausible reasoning paths, reducing the influence of an individual route or noisy subgoal.

4 NOISE-RESILIENT KNOWLEDGE GRAPH EVOLUTION

We present EvoKG, an incremental workflow for constructing and updating knowledge graphs from unstructured text. It supports both cold-start construction from an empty graph and warm-start evolution of an existing graph, and can be applied repeatedly as new document batches arrive. We operate on an attributed KG that extends standard representations with confidence-aware node properties and time-stamped edges. In particular, slowly changing node-level properties are modeled separately from time-varying relations, which are represented as temporal edges. This design addresses two key challenges:

1. Competing Values for Node-level Properties. Certain slowly changing properties (e.g., birth date) are represented at the node level. Instead of overwriting conflicting facts in prior approaches [18], we model exclusive node-level properties via a map $\pi : \mathcal{V} \rightarrow \mathcal{P}$ that maintains multiple candidates with confidence scores and contextual information, enabling robustness to noise and

context-aware reasoning. For each exclusive node-level property r , the candidate set is

$$\pi(v)[r] = \{(o_1, C(o_1), \text{ctx}_1), \dots, (o_k, C(o_k), \text{ctx}_k)\},$$

where o_i is a candidate value, $C(o_i) \in [0, 1]$ is its confidence, and ctx_i records its provenance and temporal context.

2. Temporal Evolution of Relations. Time-varying relations, including time-sensitive exclusive relations and relations with multiple coexisting values, are represented as temporal edges. We model these using **time-stamped edges** with validity intervals: $e = (v, r, v', [\tau_{\text{start}}, \tau_{\text{end}}]) \in \mathcal{E}$, allowing the KG to capture the temporal dynamics of facts. For instance, the entity Obama may simultaneously hold (Obama, works as, Senator, [2004, 2009]) and (Obama, works as, President, [2009, 2017]).

As shown in Figure 2, given a document batch $\mathcal{D}_t = \{d_1, \dots, d_N\}$ and the current KG $\mathcal{G}_t$, we extract candidate triples and incrementally align and merge them:

$$d_i \xrightarrow{f_{\text{extract}}} \hat{\mathcal{G}}_i \xrightarrow{f_{\text{merge}}} (\Delta\mathcal{V}_i, \Delta\mathcal{E}_i) \xrightarrow{\text{insert}} \mathcal{G}_{t+1}.$$

Setting $\mathcal{G}_0 = \emptyset$ yields cold-start construction; otherwise, the same recurrence evolves an existing KG. Applying it to successive batches $\mathcal{D}_1, \mathcal{D}_2, \dots$ produces a sequence $\mathcal{G}_0, \mathcal{G}_1, \mathcal{G}_2, \dots$ without retraining the reasoning model. The update process consists of (1) **entity contextualization** and (2) **relation evolution**. In the evaluated implementation, extraction and semantic alignment are instantiated with the prompts in Appendix E; vector retrieval supplies candidate matches, and the merge rules below determine the graph mutation. In Section 4.3, we discuss the scalability of this procedure.

4.1 Entity Contextualization

We handle exclusive node-level properties with multiple candidate values and associated confidence. Accordingly, entity contextualization focuses on aligning extracted entities to KG nodes and merging their properties in a noise-robust manner.

1. Contextual Alignment: Each extracted entity, $\hat{v} \in \hat{\mathcal{V}}$, is embedded using its type, name, and description, and matched to existing KG nodes via similarity search:

$$\begin{aligned} \mathbf{h}_{\hat{v}} &= \text{Enc}(\tau(\hat{v}), \text{name}(\hat{v}), \text{desc}(\hat{v})), \\ \mathcal{A}_i &= \text{TopK}_k(\{v \in \mathcal{V} \mid \text{sim}(\mathbf{h}_{\hat{v}}, \mathbf{h}_v)\}). \end{aligned}$$

A contextual scoring function $s(\mathbf{h}_{\hat{v}}, \mathbf{h}_v) \in [0, 1]$ reranks candidates $v \in \mathcal{A}_i$, and selects the best match: $v^* = \arg \max_{v \in \mathcal{A}(\hat{v})} s(\mathbf{h}_{\hat{v}}, \mathbf{h}_v)$. If $s(\mathbf{h}_{\hat{v}}, \mathbf{h}_{v^*}) < \theta$, then $\hat{v}$ is inserted as a new node.

2. Contextual Merging: For aligned entities v^* , we merge extracted properties $\hat{p} = (r, o, \text{ctx})$ into the KG. For exclusive node-level properties, we retain multiple candidate values with confidence scores based on frequency and temporal recency:

$$\begin{aligned} \pi(v^*)[r] &\leftarrow \pi(v^*)[r] \cup \{(o, C(o), \text{ctx})\}, \\ C(o) &= \frac{w(o)}{\sum_{o'} w(o')}, \quad w(o) = f(o)e^{-\gamma \Delta t(o)}. \end{aligned}$$

where $f(o)$ is the extraction frequency, $\Delta t(o) \geq 0$ is the elapsed time since o was last observed, and $\gamma \geq 0$ controls temporal decay. Thus, older unsupported observations receive lower weight; repeated evidence can offset age. The normalization makes $C(o) \in [0, 1]$ across candidates for the same exclusive relation. Each value

o is associated with context $\text{ctx} \in \mathcal{C}$, such as its document span, region, or temporal window. Values are preserved rather than immediately overwritten. In the Jordan Kessler example (Figure 2), a birth year observed seven times receives more support than a competing year observed three times, all else equal.

4.2 Relation Evolution

To capture time-varying relations, including relations that are exclusive at a given time, we represent them as time-stamped edges with validity intervals. Relation evolution therefore focuses on aligning extracted relations to the KG schema and inserting temporally grounded edges that preserve the evolution of facts.

1. Relation Normalization: We first normalize relation types by matching relation schemas. Each candidate relation schema $\hat{s} = (\tau(\hat{v}), \phi(\hat{r}), \tau(\hat{v}'))$ is embedded as $\mathbf{h}_{\hat{s}} = \text{Enc}(\hat{s})$. We compare it against KG schema embeddings $\{\mathbf{h}_{s_r}\}_{r \in \mathcal{R}}$ and select the best match: $r^* = \arg \max_{r' \in \mathcal{R}} \text{sim}(\mathbf{h}_{\hat{s}}, \mathbf{h}_{s_{r'}})$. We treat $\phi(\hat{r}) \equiv \phi(r^*)$ as a synonym match when $\max s(\mathbf{h}_{\hat{s}}, \mathbf{h}_{s_{r^*}}) > \theta$, where θ is a preset threshold.

2. Temporal Merging: Assuming the alignments $\hat{v} \rightarrow v$ and $\hat{v}' \rightarrow v'$ (from Section 4.1) are complete, we form the candidate edge $e = (v, r^*, v', [\tau_{\text{start}}, \tau_{\text{end}}])$, where the temporal range is derived from the extraction context. We insert this temporal edge so that temporally distinct facts are retained.

4.3 Update Complexity

Let n and m denote the numbers of entities and relations extracted from a document batch, and let $N = |\mathcal{V}_t|$ and $M = |\mathcal{E}_t|$. Extraction and graph mutation are linear in the extracted content, $O(n+m)$. With approximate nearest-neighbor indexes for entity and relation-schema alignment, candidate retrieval requires $O(n \log N + m \log M)$. The resulting end-to-end update complexity is therefore $O(n \log N + m \log M)$, and only the affected nodes and edges are mutated. Our prompt-based instantiation uses a constant number of LLM operation types for extraction, alignment, and merging; batching allows their calls to scale with the incoming documents rather than the full KG size.

5 EXPERIMENTS

We evaluate the prompt-based implementation of our workflow on temporal QA benchmarks and an end-to-end setting that measures the transition from an original KG to a document-enriched KG. Our experiments answer four questions: **Q1:** *How effective is EvoREASONER at multi-hop reasoning over a given temporal KG?* **Q2:** *How effectively does EvoKG acquire structured facts from raw documents?* **Q3:** *How do reasoning and KG evolution contribute to end-to-end QA performance, and how do they interact?* and **Q4:** *Which components and inference budgets drive the gains, and what is the efficiency cost?*

Datasets. Temporal KGQA. We assess multi-hop temporal reasoning on two KGQA benchmarks. TimeQuestions [9] is a factoid QA dataset over Wikidata whose answers must be grounded in specific time periods. MultiTQ [3] contains questions with diverse temporal granularities and multi-hop constraints. We randomly sample 5,000 instances and split them into five test sets. **End-to-End QA with KG Evolution.** We use two CRAG domains [34]: Movie, based on IMDB and covering films, actors, and awards;

Table 1: Temporal KGQA results (accuracy, %). RAG is not applicable since the datasets do not contain documents.

Datasets →		TimeQuestions					MultiTQ				
Methods↓		DeepSeek V3 671B	Qwen2.5 72B	LLaMA3.3 70B	GPT-4o mini	LLaMA3.1 8B	DeepSeek V3 671B	Qwen 2.5 72B	LLaMA3.3 70B	GPT-4o mini	LLaMA3.1 8B
LLM-Only	IO	54.36 $\pm$ 0.57	36.31 $\pm$ 0.16	48.56 $\pm$ 0.37	35.73 $\pm$ 0.28	28.04 $\pm$ 0.26	5.20 $\pm$ 0.90	0.50 $\pm$ 0.17	4.20 $\pm$ 0.46	1.37 $\pm$ 0.45	2.92 $\pm$ 0.48
	CoT	50.55 $\pm$ 0.54	44.84 $\pm$ 0.27	53.41 $\pm$ 0.08	41.99 $\pm$ 0.11	38.68 $\pm$ 0.21	2.83 $\pm$ 0.40	0.94 $\pm$ 0.42	4.22 $\pm$ 0.71	5.10 $\pm$ 0.20	6.42 $\pm$ 0.56
	SC	49.49 $\pm$ 2.23	44.30 $\pm$ 0.15	53.80 $\pm$ 0.20	42.16 $\pm$ 0.30	39.98 $\pm$ 0.33	2.43 $\pm$ 0.71	0.76 $\pm$ 0.29	4.34 $\pm$ 0.40	5.03 $\pm$ 0.40	6.12 $\pm$ 0.90
KG-Based	1-hop KG	37.25 $\pm$ 1.05	26.97 $\pm$.31	42.27 $\pm$ 0.09	31.55 $\pm$ 0.16	33.75 $\pm$ 0.27	7.57 $\pm$ 0.95	7.14 $\pm$ 1.22	8.28 $\pm$ 0.99	5.87 $\pm$ 1.16	5.06 $\pm$ 0.35
	RoG	39.19 $\pm$ 0.28	27.29 $\pm$ 0.48	31.21 $\pm$ 0.14	33.07 $\pm$ 0.29	18.63 $\pm$ 2.84	4.58 $\pm$ 0.41	2.06 $\pm$ 0.24	4.26 $\pm$ 0.22	4.13 $\pm$ 0.67	1.42 $\pm$ 0.33
	ToG	32.83 $\pm$ 2.24	33.07 $\pm$ 0.23	48.89 $\pm$ 0.27	32.19 $\pm$ 0.59	16.50 $\pm$ 0.52	28.90 $\pm$ 3.66	23.35 $\pm$ 1.76	25.86 $\pm$ 0.75	19.57 $\pm$ 2.80	3.90 $\pm$ 0.41
	PoG	35.16 $\pm$ 0.91	32.28 $\pm$ 0.11	48.58 $\pm$ 0.16	32.27 $\pm$ 0.26	27.58 $\pm$ 0.15	29.90 $\pm$ 2.81	25.16 $\pm$ 0.80	28.08 $\pm$ 1.04	19.77 $\pm$ 3.69	11.04 $\pm$ 0.58
	TimeR ⁴	60.19 $\pm$ 0.33	56.71 $\pm$ 0.25	56.66 $\pm$ 0.17	56.82 $\pm$ 0.11	42.15 $\pm$ 2.60	37.13 $\pm$ 0.48	35.55 $\pm$ 0.10	33.57 $\pm$ 0.09	32.47 $\pm$ 0.49	24.54 $\pm$ 0.12
EvoREASONER		67.25 $\pm$ 1.90	67.76 $\pm$ 0.28	68.63 $\pm$ 0.27	65.42 $\pm$ 0.20	56.13 $\pm$ 0.12	39.07 $\pm$ 2.26	39.45 $\pm$ 0.38	35.30 $\pm$ 1.18	34.03 $\pm$ 2.27	29.15 $\pm$ 2.15

and Sports, covering basketball and soccer matches across major leagues. Each CRAG question is paired with ground-truth answers, a web document corpus, and a mock domain-specific KG. CRAG provides the documents as one collection rather than as chronologically partitioned arrival batches; accordingly, this benchmark exposes one measurable evolution step from the original KG to the enriched KG. The same update recurrence supports cold-start construction and repeated incremental evolution when timestamped document batches are available. The provided KG contains limited, domain-specific information (e.g., basic attributes such as actors, release dates, or teams), while many questions require additional or more recent knowledge. For instance, answering “the top-3 highest-grossing movies of a studio” requires the latest aggregated financial data beyond the KG. This setting evaluates how external documents can augment the KG to support downstream reasoning.

Baselines. We use five LLMs and compare our approach against nine baselines grouped into three categories: (1) **LLM-only methods**, including IO Prompt (IO) [1], Chain-of-Thought (CoT) [32], Self-Consistency (SC) [31], and Retrieval-Augmented Generation (RAG) [15]; (2) **KG-enhanced methods**, including 1-hop KG [34] (augmenting the LLM with facts from 1-hop KG neighbors of the topic entities) and RAG + 1-hop KG [34]; and (3) **Multi-hop graph reasoning methods** previously evaluated on static KGs, Reasoning-on-Graph (RoG) [19], Think-on-Graph (ToG) [27], and Plan-on-Graph (PoG) [2]. We additionally include TimeR⁴ [24] as a fixed-KG temporal comparator on temporal KGQA¹. We report the average and standard deviation over 5 runs for all models.

5.1 Experimental Results

Q1: Effectiveness of Multi-hop Temporal Reasoning. Table 1 summarizes results on temporal QA benchmarks. Across all datasets and model scales, EvoREASONER consistently outperforms strong baselines. It achieves up to **14.0%** and **4.6%** absolute accuracy improvement over the best baseline in TimeQuestions and MultiTQ, respectively. Moreover, the performance improves further with

larger LLMs, indicating that more capable models better follow EvoREASONER’s decomposition strategy, rank relevant relations, and explore local graph neighborhoods.

On TimeQuestions, EvoREASONER improves absolute accuracy by 12.9–23.3% compared to the best LLM-only baseline, and by 7.1–14.0% over the best KG-based baseline. For example, on Qwen 2.5–72B, our method improves accuracy from 44.84% (best LLM-only) to 67.76% (+22.9%) and from 56.71% (best KG-based method) to 67.76% (+11.1%). Similarly, on LLaMA 3.1-8B, EvoREASONER improves accuracy from 39.98% (LLM-only) and 42.15% (KG-based) to 56.13%, yielding absolute gains of +16.2% and +14.0%, respectively.

On the more challenging MultiTQ dataset, EvoREASONER maintains its lead despite lower overall accuracy under higher temporal ambiguity. It improves absolute accuracy by 22.7–38.5% compared to the best LLM-only baseline and by 1.56–4.61% over the strongest KG-based baseline in each model column.

Q2: Fact Coverage of KG Construction. Before evaluating downstream QA, we measure whether EvoKG can acquire structured facts from raw text. We use the MINE benchmark [21], which contains 106 document corpora with manually verified facts, and report the fraction of verifiable facts captured by the constructed KG. As shown in Table 3, EvoKG captures 71.36% of verifiable facts, outperforming KGGen by 4.90% and substantially improving over OpenIE and GraphRAG. This result reports fact acquisition coverage on MINE. Precision, temporal-interval accuracy, alignment quality, and competing-value resolution are complementary update-quality evaluations for future work.

Q3: Reasoning and Single-stage KG Enrichment. Table 2 compares QA accuracy over the original and evolved KGs, separating performance over the initial graph from the observed change after one document-to-KG enrichment step. CRAG provides one document collection rather than chronological batches, so this analysis evaluates single-stage enrichment rather than sustained multi-batch evolution. On the **original KG**, EvoREASONER has the highest reported accuracy across domains and model scales. For example, on the Movie dataset with GPT-4o-mini, EvoREASONER achieves

¹TimeR⁴ assumes a fixed KG and is therefore evaluated only on the temporal KGQA benchmarks, not in the CRAG document-to-KG enrichment setting.

Table 2: End-to-end experiment results (accuracy, %) across LLMs of different scales. The shared evolved KG is constructed once from the CRAG document collection using LLaMA 3.3–70B; each row reports downstream QA over the original or evolved graph.

Datasets →		Movie					Sports				
Methods↓		DeepSeek V3 671B	Qwen2.5 72B	LLaMA3.3 70B	GPT-4o mini	LLaMA3.1 8B	DeepSeek V3 671B	Qwen2.5 72B	LLaMA3.3 70B	GPT-4o mini	LLaMA3.1 8B
LLM-Only	IO	56.22±0.10	34.27±0.32	51.75±0.30	38.76±0.31	34.16±0.31	38.25±0.36	23.78±0.20	28.55±0.56	26.64±0.23	18.60±0.27
	CoT	56.52±0.37	37.27±0.43	59.75±0.52	45.37±0.20	40.46±0.59	44.63±0.62	29.30±0.13	32.15±0.61	32.01±0.23	26.96±0.79
	SC	57.05±0.37	37.52±0.63	59.15±0.54	45.49±0.47	43.61±1.99	45.48±0.94	29.44±0.44	33.27±0.51	32.09±0.36	26.49±0.79
	RAG	32.39±0.61	29.13±0.27	30.94±0.44	27.55±0.20	27.04±0.38	39.02±0.62	34.53±0.38	34.91±0.27	33.02±0.14	29.35±0.31
Original KG	1-hop KG	25.13±0.35	15.75±0.13	16.35±0.10	18.29±0.20	17.59±0.20	32.71±1.24	12.24±0.13	16.21±0.59	15.65±0.40	10.49±0.12
	RAG + 1-hop KG	33.10±1.06	31.86±0.13	31.08±0.65	28.50±0.31	27.54±0.30	36.92±1.21	33.78±0.21	32.99±0.51	30.37±0.23	28.46±0.35
	RoG	40.53±0.64	18.26±1.43	42.57±0.31	28.85±0.92	14.30±0.84	41.20±1.18	16.26±3.91	31.31±0.37	33.72±0.88	13.08±1.20
	ToG	23.42±0.51	20.53±1.02	33.98±0.99	20.83±0.37	15.36±0.93	9.89±0.36	8.69±0.51	11.73±1.16	9.42±0.67	2.20±0.27
	PoG	21.83±0.45	15.19±0.49	32.71±0.63	12.86±0.27	17.88±0.48	15.26±0.94	8.27±0.46	15.65±0.87	12.38±0.47	4.63±0.35
	EvoREASONER	61.77±1.45	50.55±0.51	58.90±1.90	52.45±1.44	45.80±0.29	53.79±0.59	40.14±0.63	42.48±0.98	44.39±0.84	33.36±1.08
Evolved KG	1-hop KG	21.47±1.27	20.18±0.12	23.36±0.22	20.18±0.18	22.51±0.26	13.40±3.00	5.75±0.13	7.66±0.20	8.96±0.27	4.86±0.26
	RAG + 1-hop KG	34.28±0.51	33.91±0.20	34.97±0.27	28.97±0.20	30.80±0.18	31.62±1.10	29.63±0.10	32.80±0.27	28.50±0.40	26.45±0.30
	RoG	36.87±0.37	19.36±1.34	34.51±0.75	31.21±0.10	23.96±1.17	28.97±0.84	16.78±0.95	19.21±0.56	25.47±1.02	15.51±2.28
	ToG	25.09±1.17	31.82±0.34	40.21±0.51	24.60±0.18	23.65±1.37	24.92±2.86	23.55±0.56	22.29±0.27	15.81±1.41	10.28±1.10
	PoG	23.78±0.45	31.19±0.40	41.17±0.46	24.54±0.54	24.71±0.87	28.04±1.98	23.46±0.51	28.08±0.85	16.04±1.05	13.46±0.97
	EvoREASONER	63.13±0.10	57.84±0.70	64.85±0.71	57.52±0.53	51.11±0.79	55.61±0.33	47.90±0.23	51.17±0.62	51.64±1.98	37.06±1.58

Table 3: Verifiable-fact coverage of constructed KGs on MINE.

Method	OpenIE	GraphRAG	KGGen	EvoKG
Facts captured	29.36%	47.08%	66.46%	71.36%

52.45% accuracy, compared to 12.86% for PoG and 28.50% for RAG+1-hop KG. This comparison establishes the reported reasoning performance before document enrichment. For EvoREASONER, the evolved KG improves accuracy by 1.36–8.69% across the reported domain-model pairs. For instance, it improves from 52.45% to 57.52% on Movie (GPT-4o-mini) and from 33.36% to 37.06% on Sports (LLaMA 3.1–8B). Changes for the other multi-hop methods are heterogeneous: several improve by larger margins, whereas RoG decreases in multiple domain-model pairs after enrichment. These results characterize the performance of each reasoning method on the shared original and evolved graphs. Contribution-level attribution of individual update operations requires additional controls.

5.2 Ablation Studies on EvoREASONER

Q4: Component and Search-Budget Effects. We conduct studies on TimeQuestions and Sports, containing knowledge up to Mar. 2024, to assess the contribution of components in EvoREASONER. All results are reported using LLaMA 3.3 (70B) and LLaMA 3.1 (8B), both with a knowledge cutoff of Dec. 2023, to isolate the effect of reasoning over knowledge in KGs (additional results using different LLMs are available in the Appendix C.3). Table 4 shows that removing any sub-component leads to significant accuracy drops:

Table 4: Ablation studies (accuracy, %).

Datasets →		TimeQuestions		Sports	
Methods↓		LLaMa 3.3 70B	LLaMa 3.1 8B	LLaMa 3.3 70B	LLaMa 3.1 8B
EvoREASONER		68.63±0.27	56.13±0.12	51.17±0.62	37.06±1.58
w/o Route Decomposition		67.60±0.55	43.98±0.61	47.90±0.76	26.52±1.27
w/o Global Initialization		64.12±0.30	52.35±0.79	42.46±0.84	32.67±1.62
w/o Local Exploration		67.58±0.29	54.57±0.65	50.79±1.05	35.78±1.16

(1) removing multi-route decomposition causes the largest degradation (up to 12%), confirming the importance of soft reasoning guidance for reasoning over KGs; (2) without global initialization, performance drops by 4–9%, showing the need for accurate entity grounding; and (3) removing temporally informed local exploration consistently leads to about a 1% aggregate drop.

Route and Search-Budget Sensitivity. Figure 3 further examines how route diversity affects performance on the evolved Sports KG with LLaMA 3.1–8B. Accuracy increases as additional routes are considered, showing that alternative decompositions help recover answers when a single path is broken by missing links or noisy neighborhoods. Table 5 varies local search depth d and width w . Increasing width produces the clearest gains, improving from 44.55% at ($w = 1, d = 2$) to 51.99% at ($w = 30, d = 2$), while deeper search beyond two hops yields little additional benefit. This supports our

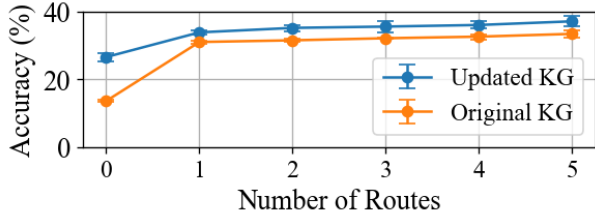


Figure 3: Route-wise QA accuracy on the evolved Sports KG.

Table 5: Search depth (d) and width (w) sensitivity on the evolved Sports KG.

	$d = 1$	$d = 2$	$d = 3$
$w = 1$	42.99 $\pm$ 0.40	44.55 $\pm$ 0.36	44.78 $\pm$ 0.82
$w = 5$	43.38 $\pm$ 11.07	49.84 $\pm$ 2.01	47.43 $\pm$ 4.25
$w = 10$	43.81 $\pm$ 10.68	49.69 $\pm$ 0.94	50.16 $\pm$ 3.84
$w = 20$	46.82 $\pm$ 1.21	51.34 $\pm$ 0.62	51.25 $\pm$ 0.71
$w = 30$	50.70 $\pm$ 0.62	51.99 $\pm$ 0.13	51.64 $\pm$ 0.79

Table 6: Reasoning-stage efficiency on the evolved Movie KG.

Method	Latency (sec)	Accuracy (%)	Memory (MB)	Tokens	Calls
IO	12.97 $\pm$ 8.98	34.16 $\pm$ 0.31	301	153.69 $\pm$ 0.23	1
RAG	21.33 $\pm$ 13.65	27.04 $\pm$ 0.38	364	2195.01 $\pm$ 1.50	1
ToG	190.80 $\pm$ 8.93	23.65 $\pm$ 1.37	397	5910.27 $\pm$ 45.37	21.8
PoG	191.49 $\pm$ 8.01	24.71 $\pm$ 0.87	836	5874.18 $\pm$ 35.24	37.1
EvoREASONER	71.46 $\pm$ 83.34	51.11 $\pm$ 0.79	739	2365.44 $\pm$ 535.71	26.5

design choice of broad, temporally guided local exploration rather than unconstrained deep traversal.

5.3 Efficiency and Scalability Analysis

Q4: Efficiency. Table 6 reports per-query latency, accuracy, memory usage, token consumption, and LLM-call counts on the evolved Movie KG using LLaMA 3.1-8B with matched decoding settings. EvoREASONER is slower than prompting-only and RAG baselines because it performs route planning and KG-grounded exploration, but it is much more accurate. Compared with multi-hop graph reasoning baselines, EvoREASONER reaches 51.11% accuracy while taking 71.46 seconds per query, whereas ToG and PoG take about 191 seconds and reach only 23.65% and 24.71% accuracy. Although EvoREASONER uses multiple LLM calls, its total token consumption (2365.44 tokens per query) is lower than ToG and PoG (5910.27 and 5874.18 tokens), because it performs bounded route planning and avoids repeated step-by-step graph traversal prompts. This corresponds to roughly a 2.6 $\times$ latency reduction and about a 2.5 $\times$ token reduction relative to these multi-hop graph baselines, while more than doubling accuracy in this setting.

For KG evolution, detailed latency and token-cost scaling curves are reported in Appendix C.4. The reported curves show increasing latency and token usage with document length under the API-mediated evaluation setup.

6 RELATED WORK

KG Reasoning. Reasoning over knowledge graphs (KGs) involves drawing structured inferences to support tasks like question answering and decision-making [23]. Existing methods typically fall into two categories: (1) embedding-based approaches and (2) LLM-driven approaches. Embedding-based methods encode entities and relations into dense vectors to retrieve relevant subgraphs. Recent work such as SR [36], UniKGQA [13], and ReasoningLM [11] adopt this strategy, but may struggle with noise and limited interpretability. LLM-driven approaches leverage large language models to guide or perform reasoning. StructGPT [10] generates structured queries, RoG [19] performs iterative reasoning directly on graph structures, ToG [27] identifies salient subgraphs, and PoG [2] decomposes queries into multi-step plans. KG-Agent [12] extends this with tool-augmented traversal. However, most approaches assume a static KG and treat temporal attributes as metadata, limiting their ability to handle evolving knowledge.

KG Extraction. Traditional KG extraction techniques often rely on rule-based extraction from curated resources (e.g., YAGO [26]), but these approaches are brittle and hard to scale. More recent efforts turn to LLMs for end-to-end KG extraction from raw text. Techniques such as JointIE [25], EXTRACT [35], and UP-KG [5] formulate KG extraction as a text-to-graph generation task. These techniques demonstrate promising performance but often overwrite existing knowledge without accounting for temporal drift or contradictions. Our work builds on this line of work but introduces a temporal-aware, contradiction-tolerant evolution process, allowing the KG to store multiple values, track confidence, and maintain historical facts.

7 CONCLUSION

We presented a structured workflow for LLM reasoning over evolving knowledge graphs. Its two components connect three reusable principles: multi-route planning under graph incompleteness, context-informed temporal refinement, and incremental evolution under conflicting updates. Across temporal KGQA and end-to-end document-to-graph QA, the evaluated prompt-based implementation improves consistently over LLM-only, retrieval, and graph-reasoning baselines. The CRAG evaluation measures the observable transition enabled by its single document collection, while EvoKG supports cold-start construction and repeated evolution over chronologically arriving batches. These results show that evolving external knowledge can improve an LLM’s knowledge-usage efficiency while retaining its underlying reasoning model. Future work can evaluate longer update sequences, direct contradiction-resolution labels, and question-type-specific temporal gains on benchmarks with chronologically segmented corpora.

REFERENCES

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models Are Few-Shot Learners. *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), 1877–1901.
- [2] Liyi Chen, Panrong Tong, Zhongming Jin, Ying Sun, Jieping Ye, and Hui Xiong. 2024. Plan-on-Graph: Self-Correcting Adaptive Planning of Large Language Model on Knowledge Graphs. In *Advances in neural information processing systems (NeurIPS)*.
- [3] Ziyang Chen, Jinzhi Liao, and Xiang Zhao. 2023. Multi-granularity temporal question answering over knowledge graphs. In *Proceedings of the 61st Annual*

- Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 11378–11392.
- [4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
 - [5] Shahenda Hatem, Ghada Khoriba, Mohamed H Gad-Elrab, and Mohamed ElHelw. 2024. Up To Date: Automatic Updating Knowledge Graphs Using LLMs. *Procedia Computer Science* 244 (2024), 327–334.
 - [6] Zhen Huang, Zengzhi Wang, Shijie Xia, Xuefeng Li, Haoyang Zou, Ruijie Xu, Run-Ze Fan, Lyumanshan Ye, Ethan Chern, Yixin Ye, et al. 2024. OlympicArena: Benchmarking multi-discipline cognitive reasoning for superintelligent AI. *Advances in Neural Information Processing Systems (NeurIPS)* 37 (2024), 19209–19253.
 - [7] IBM. 2024. *slate-125m-english-rtrvr model card*. Accessed: 2025-05-13. <https://dataplatfrom.cloud.ibm.com/docs/content/wsj/analyze-data/fm-slate-125m-english-rtrvr-v2-model-card.html?context=wx>
 - [8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.
 - [9] Zhen Jia, Soumajit Pramanik, Rishiraj Saha Roy, and Gerhard Weikum. 2021. Complex temporal question answering on knowledge graphs. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 792–802.
 - [10] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. 2023. StructGPT: A General Framework for Large Language Model to Reason over Structured Data. In *Conference on Empirical Methods in Natural Language Processing*. 9237–9251.
 - [11] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yaliang Li, and Ji-Rong Wen. 2023. ReasoningLM: Enabling Structural Subgraph Reasoning in Pre-trained Language Models for Question Answering over Knowledge Graph. In *Conference on Empirical Methods in Natural Language Processing*. 3721–3735.
 - [12] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2025. KG-Agent: An Efficient Autonomous Agent Framework for Complex Reasoning over Knowledge Graph. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 9505–9523. doi:10.18653/v1/2025.acl-long.468
 - [13] Jinhao Jiang, Kun Zhou, Xin Zhao, and Ji-Rong Wen. 2023. UniKGQA: Unified Retrieval and Reasoning for Solving Multi-hop Question Answering Over Knowledge Graph. In *The Eleventh International Conference on Learning Representations*.
 - [14] Amanda Kau, Xuzeng He, Aishwarya Nambissan, Aland Astudillo, Hui Yin, and Amir Aryani. 2024. Combining knowledge graphs and large language models. *arXiv preprint arXiv:2407.06564* (2024).
 - [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), 9459–9474.
 - [16] Aixian Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
 - [17] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Advances in Neural Information Processing Systems (NeurIPS)* 35 (2022), 2507–2521.
 - [18] Yifan Lu, Yigeng Zhou, Jing Li, Yequan Wang, Xuebo Liu, Daojing He, Fangming Liu, and Min Zhang. 2025. Knowledge editing with dynamic knowledge graphs for multi-hop question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24741–24749.
 - [19] Linhao Luo, Yuan-Fang Li, Reza Haf, and Shirui Pan. 2024. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In *International Conference on Learning Representations (ICLR)*.
 - [20] Yu. A. Malkov and Dmitry A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836. doi:10.1109/TPAMI.2018.2889473
 - [21] Belinda Mo, Kyssen Yu, Joshua Kazdan, Proud Mpala, Lisa Yu, Charilaos Kanatsoulis, and Sanmi Koyejo. 2025. KGGen: Extracting Knowledge Graphs from Plain Text with Language Models. In *Advances in Neural Information Processing Systems*, Vol. 38. https://proceedings.neurips.cc/paper_files/paper/2025/hash/2b36845e832d2b1a60bcad8c4c6481f-Abstract-Conference.html
 - [22] OpenAI. 2024. *GPT-4o mini: advancing cost-efficient intelligence*. Accessed: 2025-05-13. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
 - [23] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3599.
 - [24] Xinying Qian, Ying Zhang, Yu Zhao, Baohang Zhou, Xuhui Sui, Li Zhang, and Kehui Song. 2024. TimeR4: Time-aware retrieval-augmented large language models for temporal knowledge graph question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 6942–6952.
 - [25] Bo Qiao, Zhuoyang Zou, Yu Huang, Kui Fang, Xinghui Zhu, and Yiming Chen. 2022. A joint model for entity and relation extraction based on BERT. *Neural Computing and Applications* (2022), 1–11.
 - [26] Fabian M Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2007. Yago: a core of semantic knowledge. In *Proceedings of the 16th international conference on World Wide Web*. 697–706.
 - [27] Jiahuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *International Conference on Learning Representations (ICLR)*.
 - [28] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A large language model for science. *arXiv preprint arXiv:2211.09085* (2022).
 - [29] Neng Wang, Hongyang Yang, and Christina Wang. 2023. FinGPT: Instruction Tuning Benchmark for Open-Source Large Language Models in Financial Datasets. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*.
 - [30] Song Wang, Yaochen Zhu, Haochen Liu, Zaiyi Zheng, Chen Chen, and Jundong Li. 2024. Knowledge editing for large language models: A survey. *Comput. Surveys* 57, 3 (2024), 1–37.
 - [31] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*.
 - [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems (NeurIPS)* 35 (2022), 24824–24837.
 - [33] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
 - [34] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Gui, Ziran Jiang, Ziyu Jiang, et al. 2024. CRAG-comprehensive RAG benchmark. *Advances in Neural Information Processing Systems (NeurIPS)* 37 (2024), 10470–10490.
 - [35] Bowen Zhang and Harold Soh. 2024. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. *arXiv preprint arXiv:2404.03868* (2024).
 - [36] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. 2022. Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5773–5784.

A SCALABILITY ANALYSIS

We show that our KG evolution is scalable to large-scale settings. The evolution pipeline is largely **decoupled from the total KG size**, with complexity dominated by the input corpus. Let n and m be the number of extracted entities and relations, and N and M be the number of nodes and edges in the KG, respectively. Besides entity and relation alignment, which use similarity search, graph mutation and insertion operate on the extracted content and take $O(n + m)$ time. LLM-mediated extraction is accounted for separately in the empirical latency and token measurements below. Entity and relation alignment use HNSW-based similarity search, which Malkov and Yashunin [20] shows has expected complexity $O(n \log N + m \log M)$ in practice.² Finally, the evaluated workflow uses a fixed set of LLM operation types for extraction, alignment, and merging; its per-document latency and token use are reported below.

²This expression follows the indexing discussion in Malkov and Yashunin [20]; its applicability depends on the index construction and data distribution.

B ADDITIONAL EXPERIMENT DETAILS

Datasets. We summarize the statistics of all datasets used in Table 7.

LLM Models. We evaluate our methods using five LLMs, ordered by parameter size: DeepSeek V3 (671B) [16], Qwen 2.5 (72B) [33], LLaMA 3.3 (70B) [4], GPT-4o-mini [22], and LLaMA 3.1 (8B) [4]. All models follow the same QA protocol for comparison. We use a standard lightweight sentence-transformers model (IBM/slate-125m-english-rtrvr-v2 [7]) to generate semantic embeddings. We performed the KG updating using the LLaMA 3.3 (70B) model. All the models are accessible through online API providers, such as OpenAI, DeepSeek, and OpenRouter.

C ADDITIONAL EXPERIMENT RESULTS

C.1 Cross-model Robustness

We examine performance across multiple LLMs of different scales and training regimes (e.g., DeepSeek-V3, Qwen2.5, LLaMA3.3, GPT-4o, and LLaMA3.1). EvoREASONER improves over the included baselines for every model in Table 1, indicating that the reported results extend across these model families and scales. The evidence in this subsection concerns cross-model robustness under the shared prompt template.

C.2 Additional Comparison with Temporal KG Methods

We include TimeR⁴ [24], a recent temporal KG reasoning method, as an additional baseline on temporal QA benchmarks (Table 1).

TimeR⁴ is designed for temporal reasoning over structured KGs, and is evaluated on datasets such as TimeQuestions and MultiTQ where temporal signals are explicitly modeled. Our results show that EvoREASONER consistently outperforms TimeR⁴ across model scales, demonstrating the effectiveness of our adaptive multi-hop reasoning framework.

We note that TimeR⁴ is not directly evaluated in the end-to-end setting (Movie and Sports), as it assumes a fixed KG and does not incorporate document-driven KG updates.

C.3 Ablation Studies on More LLMs

We additionally tested the effectiveness of components in EvoREASONER using Qwen 2.5–70B, a LLM from another family, and observed consistent trends: removing key components still caused significant performance drops, confirming the robustness of our method across LLM families.

C.4 Empirical Efficiency and Scalability Analysis

This appendix complements the update-complexity discussion in Section 4 by providing empirical measurements of runtime, memory usage, token cost, and scaling behavior for both the KG-evolution stage (EvoKG) and the KG-reasoning stage (EvoReasoner). LLM inference is accessed through public APIs; reasoning measurements use LLaMA-3.1-8B with matched decoding settings across methods.

C.4.1 Reasoning Efficiency of EvoREASONER. We report detailed runtime, memory usage, token consumption, LLM-call counts, and

accuracy measurements for the reasoning stage (EvoREASONER) and representative baselines. All methods are evaluated on the evolved MOVIE dataset using the API-hosted LLaMA-3.1-8B model with matched decoding settings.

Table 9 summarizes per-query wall-clock latency, peak memory usage, token consumption, LLM-call counts, and accuracy. EvoREASONER achieves the highest accuracy across all methods while incurring substantially lower latency and token cost than existing multi-hop graph reasoning approaches (ToG and PoG). In particular, EvoREASONER is approximately 2.6× faster than ToG and PoG, which repeatedly invoke LLM calls at each hop during graph traversal.

The additional runtime and memory overhead of EvoREASONER relative to LLM-only baselines (e.g., IO, RAG) arises from multi-route planning, entity alignment, and temporally informed local exploration. However, this overhead remains moderate and significantly lower than that of PoG, while yielding a large absolute accuracy improvement. These results demonstrate that EvoREASONER offers a favorable accuracy–efficiency trade-off for multi-hop reasoning over evolving knowledge graphs.

C.4.2 KG Evolution Efficiency.

Per-Document Extraction Latency. To assess the efficiency of KG evolution, we measure the per-document extraction time across all corpora. Figure 4 plots the KG-update latency as a function of document length (in characters). The results exhibit a clear positive trend: longer documents incur proportional increases in processing time. This is expected, as extraction latency is dominated by the LLM forward pass, whose computational cost grows with input token count. We observe no evidence of superlinear blow-up. Instead, the empirical trend aligns with the theoretical $O(n \log n + m \log m)$ complexity of EvoKG’s alignment and merge operations. Variance in the vertical direction is attributable to LLM-provider latency: extraction is executed via the OpenRouter API, whose backend routing introduces heterogeneity in response times that is unrelated to algorithmic complexity.

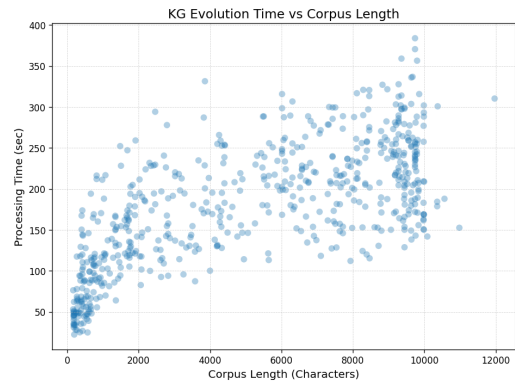


Figure 4: Observed per-document KG evolution latency as a function of corpus length (in characters). Longer documents require more extraction time under the API-mediated evaluation setup.

Table 7: Statistics of datasets.

Dataset	# Entities	# Relations	# Entity Types	# Relation Types	QA Pairs	Corpus
Movie (Original KG)	265,201	461,772	5	7	565	2,825
Movie (Evolved KG)	345,539	566,328	454	7,590	565	2,825
Sports (Original KG)	72,442	279,720	5	6	428	2,140
Sports (Evolved KG)	131,468	401,662	443	9,151	428	2,140
TimeQuestions	113,716	229,767	3	884	3,237	–
MultitQ	10,742	166,760	3	251	5,000	–

Table 8: Ablation studies on EvoREASONER.

Datasets	TimeQuestions	Sports (Original KG)	Sports (Evolved KG)
EvoREASONER	67.76$\pm$0.28	40.14$\pm$0.63	47.90$\pm$0.23
w/o Route Decomposition	56.04 $\pm$ 1.50	18.81 $\pm$ 0.98	35.95 $\pm$ 0.60
w/o Global Initialization	60.46 $\pm$ 0.35	38.63 $\pm$ 0.96	41.12 $\pm$ 1.01
w/o Local Exploration	67.05 $\pm$ 0.04	39.43 $\pm$ 1.01	46.78 $\pm$ 0.32

Table 9: Runtime, memory, token usage, and LLM calls of several baselines.

Method	Total Latency (sec)	Accuracy (%)	Average Memory Usage (MB)	Total Tokens	LLM Calls
IO	12.97 $\pm$ 8.98	34.16 $\pm$ 0.31	301	153.69 $\pm$ 0.23	1
RAG	21.33 $\pm$ 13.65	27.04 $\pm$ 0.38	364	2195.01 $\pm$ 1.50	1
ToG	190.80 $\pm$ 8.93	23.65 $\pm$ 1.37	397	5910.27 $\pm$ 45.37	21.8
PoG	191.49 $\pm$ 8.01	24.71 $\pm$ 0.87	836	5874.18 $\pm$ 35.24	37.1
EvoREASONER (Ours)	71.46 $\pm$ 83.34	51.11 $\pm$ 0.79	739	2365.44 $\pm$ 535.71	26.5

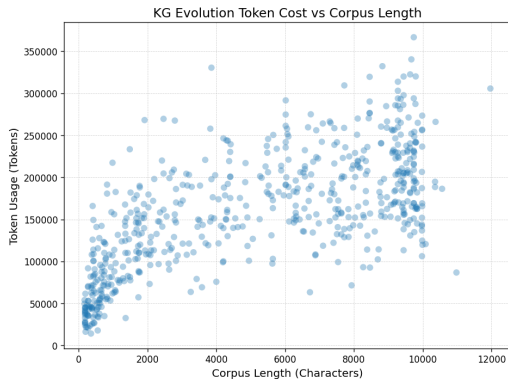


Figure 5: Token usage during KG extraction plotted against corpus length. Token cost grows approximately linearly with input size due to proportional input tokens and a fixed number of extraction calls per document. The smooth scaling pattern confirms that EvoKG’s update cost remains predictable and stable across document sizes.

To clarify practical deployment: although Figure 4 reports per-document latency, EvoKG supports data-parallel extraction. When run in parallel, the effective end-to-end KG-update time is substantially lower than the sum of individual extraction times.

Token Cost Scaling. Figure 5 presents token usage versus document length. Token consumption increases with input size, reflecting proportional input tokens together with output tokens for extracted triples and temporal metadata.

Together, Figure 4 and 5 characterize observed latency and token usage across the evaluated document lengths.

C.5 Licenses for Existing Assets

We use the following models and assets in this work, all of which are properly cited and used under their respective licenses:

- **DeepSeek V3 (671B):** Accessed via DeepSeek API, under the DeepSeek open-source model license available at <https://github.com/deepseek-ai/DeepSeek-LLM/blob/main/LICENSE-MODEL>.
- **Qwen 2.5 (72B):** Accessed via OpenRouter, based on Qwen model family released by Alibaba under a commercial-use friendly license.
- **LLaMA 3.3 & 3.1 (70B, 8B):** Accessed via OpenRouter API, follows Meta’s LLaMA 3 license available at <https://ai.meta.com/llama/license/>.
- **GPT-4o-mini:** Accessed through OpenAI API, used in accordance with the terms of service and usage policies listed at <https://openai.com/policies/terms-of-use>.
- **IBM Slate-125M-english-rtrvr-v2:** Released by IBM under Apache 2.0 License via <https://huggingface.co/ibm-granite/granite-embedding-125m-english>.

All datasets used in the paper (TimeQuestions, MultiTQ, CRAG benchmark) are publicly released by their authors for research purposes and properly cited in the paper.

Compute Resources. All experiments involving large language models were conducted via public API providers. Specifically, we accessed models through the OpenAI and OpenRouter platforms, which internally manage GPU compute infrastructure. As such, we do not control the underlying hardware but follow their standard rate limits and access conditions.

All preprocessing, knowledge graph construction, and hosting were performed on a cloud server equipped with an Intel Xeon E5-2698 CPU (40 cores, 2.2 GHz) and 500GB of main memory. Each full run of a QA benchmark (including reasoning) takes less than 2–3 hours, depending on model size and API latency. The KG update takes longer, which requires processing more than 3,400 corpora with an average length of 10,000 words, and usually can be accomplished within 24 hours. No dedicated GPU resources were used beyond the API endpoints.

D PROMPT TEMPLATES FOR EVOREASONER

D.1

Prompt Templates for Route Planning

You are a helpful assistant who is good at answering questions in the domain domain by using knowledge from an external knowledge graph. Before answering the question, you need to break down the question so that you may look for the information from the knowledge graph in a step-wise operation. Hence, please break down the process of answering the question into as few sub-objectives as possible based on semantic analysis. A query time is also provided; please consider including the time information when applicable.

There can be multiple possible route to break down the question, aim for generating route possible routes. Note that every route may have a different solving efficiency, order the route by their solving efficiency.

Return your reasoning and sub-objectives as multiple lists of strings in a flat JSON of format: "reason": "...", "routes": [[<a list of sub-objectives>], [<a list of sub-objectives>], ...]. (TIP: You will need to escape any double quotes in the string to make the JSON valid)

-Example-

Q: Which of the countries in the Caribbean has the smallest country calling code?

Query Time: 03/05/2024, 23:35:21 PT

Output: { "reason": "The most efficient route involves directly identifying Caribbean countries and their respective calling codes, as this limits the scope of the search. In contrast, routes that involve broader searches, such as listing all country calling codes worldwide before

filtering, are less efficient due to the larger dataset that needs to be processed. Therefore, routes are ordered based on the specificity of the initial search and the subsequent steps required to narrow down to the answer.",

"routes": [{"List all Caribbean countries", "Determine the country calling code for each country", "Identify the country with the smallest calling code"}, {"Identify Caribbean countries", "Retrieve their country calling codes", "Compare to find the smallest"}, {"Identify the smallest country calling code globally", "Filter by Caribbean countries", "Select the smallest among them"}, {"List all country calling codes worldwide", "Filter the calling codes by Caribbean countries", "Find the smallest one"}]}

Q: <query>

Query Time: <query time>

Output Format (flat JSON): {"reason": "...", "routes": [[<a list of sub-objectives>], [<a list of sub-objectives>], ...]}

Output:

Prompt Templates for Context-Aware Global Initialization

-Goal-

You are presented with a question in the domain domain, its query time, and a potential route to solve it.

1) Determine the topic entities asked in the query and each step in the solving route. The topic entities will be used as source entities to search through a knowledge graph for answers. It's preferable to mention the entity type explicitly to ensure a more precise search hit.

2) Extract those topic entities from the query into a string list in the format of ["entity1", "entity2", ...]. Consider extracting the entities in an informative way, combining adjectives or surrounding information.

A query time is provided - please consider including the time information when applicable.

NEVER include ANY EXPLANATION or NOTE in the output, ONLY OUTPUT JSON

-Examples-

<few-shot examples>

Question: <query>

Query Time: <query time>

Solving Route: <route>

Output Format: ["entity1", "entity2", ...]

Output:

Prompt Templates for Global Initialization Relevance Scoring

-Goal-

You are presented with a question in the domain domain, its query time, a potential route to solve it, and a list of entities extracted from a noisy knowledge graph.

The goal is to identify all possible relevant entities to answering the steps in the solving route and, therefore, answer the question.

You need to consider that the knowledge graph may be noisy and relations may split into similar entities, so it's essential to identify all relevant entities.

The entities' relevance would be scored on a scale from 0 to 1 (use at most 3 decimal places, and remove trailing zeros; the sum of the scores of all entities is 1).

-Steps-

1. You are provided a set of entities (type, name, description, and potential properties) globally searched from a knowledge graph that most similar to the question description, but may not directly relevant to the question itself.

Given in the format of "ent_i: (<entity type>: <entity name>, desc: "description", props: {key: [val_1 (70%, ctx:"context"), val_2 (30%, ctx:"context")], ...})" where "i" is the index, the percentage is confidence score, "ctx" is an optional context under which the value is valid. Each property may have only a single value, or multiple valid values of vary confidence under different context.

2. Score ***ALL POSSIBLE*** entities that are relevant to answering the steps in the solving route and therefore answering the question, and provide a short reason for your scoring.

Return its index (ent_i) and score into a valid JSON of the format: {"reason": "reason", "relevant_entities": {"ent_i": 0.6, "ent_j": 0.3, ...}}. (TIP: You will need to escape any double quotes in the string to make the JSON valid)

NEVER** include ANY EXPLANATION or NOTE in the output, **ONLY OUTPUT JSON

-Examples-

<few-shot examples>

Question: <query>

Query Time: <query time>

Solving Route: <route>

Entities: <topk entities str>

Output Format (flat JSON): {"reason": "reason", "relevant_entities": {"ent_i": 0.6, "ent_j": 0.3, ...}}

Output:

PROMPT TEMPLATES FOR EVOKG

Prompt Templates for KG Update

1. Overview

You are a top-tier algorithm designed for extracting information in structured formats to build a knowledge graph. Try to capture as much information from the text as possible without sacrificing accuracy.

Do not add any information that is not explicitly mentioned in the text. The text document will only be provided to you ONCE. After reading it, both you and we will no longer have access to it (like a closed-book exam).

Therefore, extract all self-contained information needed to reconstruct the knowledge. Do NOT use vague pronouns like "this", "that", or "it" to refer to prior context in the text. Always use full, explicit names or phrases that can stand alone.

- ****Nodes**** represent entities and concepts.

- The aim is to achieve simplicity and clarity in the knowledge graph, making it accessible to a vast audience.

2. Labeling Nodes

- ****Consistency****: Ensure you use available types for node labels. Ensure you use basic or elementary types for node labels.

- For example, when you identify an entity representing a person, always label it as ****person****. Avoid using more specific terms like 'mathematician' or 'scientist'.

- ****Node IDs****: Never utilize integers as node IDs. Node IDs should be names or human-readable identifiers found in the text.

- ****Relationships**** represent connections between entities or concepts. Ensure consistency and generality in relationship types when constructing knowledge graphs. Instead of using specific and momentary type such as 'BECAME_PROFESSOR', use more general and timeless relationship types like 'PROFESSOR'. Make sure to use general and timeless relationship types!

3. Coreference Resolution

- ****Maintain Entity Consistency****: When extracting entities, it's vital to ensure consistency. If an entity, such as "John Doe", is mentioned multiple times in the text but is referred to by different names or pronouns (e.g., "Joe", "he"),

always use the most complete identifier for that entity throughout the knowledge graph. In this example, use "John Doe" as the entity ID. Remember, the knowledge graph should be coherent and easily understandable, so maintaining consistency in entity references is crucial.

4. Strict Compliance

Adhere to the rules strictly. Non-compliance will result in termination.

-Goal-

Given a text document, identify all entities from the text and all relationships among the identified entities.

-Steps-

1. Identify all entities. For each identified entity, extract its type, name, description, and properties.

- type: One of the following types, but not limited to: [`<entity_types>`]. Please refrain from creating a new entity type, always try to fit the entity to one of the provided types first.

- name: Name of the entity, use the same language as input text. If English, capitalize the name.

- description: Comprehensive and general description (under 50 words) of the entity.

- properties: Entity properties are key-value pairs modeling special relations where an entity has **only one valid value at any point in its lifetime**. These properties **do not change frequently**.

- Each type of entity can have a distinct set of properties.

- If any properties were not mentioned in the text, please skip them.

- Only include those properties with a **valid value**.

- Example entity properties: A person-typed entity may have a birthday and nationality. A movie-typed entity may have a release date and language. What they have in common is that they tend to have one valid value at any point in their lifetime.

Format each entity as a list of 3 string elements and a set of key-value pairs: [`"type"`, `"name"`, `"description"`, {"key": `"val"`, ...}], assign this list to a key named `"ent_i"`, where `i` is the entity index.

2. Among the entities identified in step 1, identify all pairs of (`source_entity`, `target_entity`) that are **clearly related** to each other and extract their description and potential properties.

- `source_entity_name`: name of the source entity, **MUST BE** one of the entity names identified in step 1 (the `"name"`).

- `relation_name`: up to **three words** as a predicate describing the general relationship between the source entity and target entity, capitalized and joined with underscores (e.g., [`<relation_types>`]).

- `target_entity_name`: name of the target entity, **MUST BE** one of the entity names identified in step 1 (the `"name"`).

- description: short and concise explanation as to why you think the source entity and the target entity are related to each other

- relation_properties: Relation properties are special complement parts of relations, they store information that is not manifest by the relation name alone.

- Each type of relation can have a distinct set of properties.

- Example relation properties: A `WORK_IN` relation may have an occupation. A `HAS_POPULATION` relation may have the value of the population.

Format each relationship as a list of 4 string elements and a set of key-value pairs: [`"source_entity_name"`, `"relation_name"`, `"target_entity_name"`, `"description"`, {"key": `"val"`, ...}], assign this list to a key named `"rel_i"`, where `i` is the relation index.

To better extract relations, please follow these two sub-steps exactly.

a. Identify **exclusive relations that evolve over time** (time-sensitive exclusivity). These relationships should be extracted as **temporal relations** instead of properties.

- If a relationship **can change over time but only one value is valid at any given moment**, it must be modeled as a **temporal relationship with timestamps**. Example relationships include:

- A person works at only one company at a time: (Person: JOHN)-[WORKS_AT, props: {valid_from: 2019-01-01, valid_until: 2021-06-01}]->(Company: IBM).

- A person resides in only one place at a time: (Person: LISA)-[LIVES_IN, props: {valid_from: 2021-03-14, valid_until: None}]->(Geo: BOSTON).

- A geographic region has a population that changes over time: (Geo: UNITED STATES)-[HAS_POPULATION, props: {valid_from: 2025, valid_until: None, population: 340.1 million}]->(Geo: UNITED STATES).

- These relationships should be formatted as a list of 4 string elements and a set of key-value pairs: [`"source_entity"`, `"relation_name"`, `"target_entity"`, `"relation_description"`, {"valid_from": `"YYYY-MM-DD"`, "valid_until": `"YYYY-MM-DD"`, "key": `"val"`, ...}].

b. Identify **accumulative relations** (non-exclusive relationships). These relations **do not need deprecation** and can have multiple values coexisting. Example relationships include:

- Actors can act in multiple movies: (Person: AMY)-[ACTED_IN, props: {character: Anna, year: 2019}]->(Movie: A GOOD MOVIE).

- A person can have multiple skills: (Person: AMY)-[HAS_SKILL, props: {skill: jogging}]->(Person: AMY).

- A person can have multiple friends: (Person: JENNY)-[HAS_FRIEND]->(Person: AMY).

- Format these relations as: [`"source_entity"`, `"relation_name"`, `"target_entity"`, `"relation_description"`, {"key": `"val"`, ...}].

3. Return output as a flat JSON. **NEVER** include ANY EXPLANATION or NOTE in the output, **ONLY OUTPUT FLAT JSON**

You must attempt to extract as many entities and relations as you can. It's fine to infer entity roles and connections when strongly suggested by context or scene

description.

But it's crucial that "source_entity_name" and "target_entity_name" in the identified relations, *MUST BE* one of the identified entity names.

Here are some examples:

<few-shot examples>

```
{
  "text": "Marie Curie, a Polish-French physicist, was
born in Warsaw and later became a professor at the
University of Paris. She was awarded two Nobel Prizes:
one in Physics (1903) and one in Chemistry (1911).",
  "output": {
    "ent_0": ["Person", "Marie Curie", "Polish-French
physicist", {"nationality": "Polish-French", "birth_place":
"Warsaw"}],
    "ent_1": ["Geo", "Warsaw", "Marie Curie's birthplace",
{}],
    "ent_2": ["Organization", "University of Paris", "Where
Marie Curie was a professor", {}],
    "ent_3": ["Award", "Nobel Prize in Physics", "Award
won by Marie Curie in 1903", {"year": "1903", "type": "Nobel
Prize"}],
    "ent_4": ["Award", "Nobel Prize in Chemistry", "Award
won by Marie Curie in 1911", {"year": "1911", "type": "Nobel
Prize"}],
    "rel_0": ["Marie Curie", "WORKS_AT", "University of
Paris", "Marie Curie worked at the University of Paris",
{"valid_from": "None", "valid_until": "None", "occupation":
"professor"}],
    "rel_1": ["Marie Curie", "WON", "Nobel Prize in
Physics", "Marie Curie won the Nobel Prize in Physics in
1903", {"year": "1903"}],
    "rel_2": ["Marie Curie", "WON", "Nobel Prize in Chem-
istry", "Marie Curie won the Nobel Prize in Chemistry in
1911", {"year": "1911"}]
  },
  "explanation": "The nationality and birth place is life-
time exclusive, so they are Marie's properties. WORKS_AT
is modeled as exclusive relations, given the valid period.
WON award is an accumulative relationship, so no valid
period is given."
}, ...
```

Text: input_text

Output format (flat JSON):

```
{
  "ent_i": ["type", "name", "description", {"key": "val", ...}],
  "rel_j": ["source_entity_name", "relation_name", "tar-
get_entity_name", "relation_description", {"key": "val", ...}],
  ...
}
```

****REMINDER**:** You are rewarded for high coverage and

precise reasoning. Extract as much useful information as you can.
Output: